

MiniZinc for Formal Reasoning^{*}

Peter J. Stuckey, October 2023

***Most of the credit goes to Henk Bierlee, Jip Dekker, Jason Nguyen, Guido Tack.
Most of the mistakes are mine.**

Overview

A **quick look** at MiniZinc

MiniZinc features

How MiniZinc is **compiled** to FlatZinc

Encoding state in MiniZinc

Future plans for MiniZinc

A quick look at MiniZinc

MiniZinc



- Constraint Modelling Language
- First version 2007, re-implemented v2.0 released 2014
- Used in industry and academia
- Thousands of downloads per month, 2 Coursera courses, ~1000 citations of 2007 paper
- Full system (compiler, IDE, tutorial and reference documentation, lots of solver backends...)
- Open source, available from www.minizinc.org

What kind of language?

- Specification language?
 - we could define **what a program means** purely in terms of logic (except recursion makes it a bit awkward)
- Constraint Logic Programming?
 - quite similar, **mixing “procedural” and purely declarative aspects** (e.g. `forall` can act like a loop or a predicate)
 - but MiniZinc leaves **all non-determinism to the solver**
- Functional Programming?
 - base language is **functional** (even kind of **pure**)

Quick MiniZinc demo

	Coding	UX	Doc	Testing	Security
T_1 	1 5	2 4	4 5	3 1	5 4
T_2 	3 1	5 1			
T_3 	5 2	4 / 3	2 / 2	1 / 4	3 / 2
T_4 	1 4	3 / 2	5 / 4	4 / 3	2 / 1
T_5 	4 3	2 / 5	3 / 1	5 / 2	1 / 5

T1 prefers Coding
most, Security least

Coding group prefers T2
most, T1 least

Find a matching of team members to groups
that is **stable**:
i.e. no member would prefer to
change to a different group,
which would also prefer them
to their current assigned member

Quick MiniZinc demo

	Coding	UX	Doc	Testing	Security
T_1 	1 / 5	2 / 4	4 / 5	3 / 1 	5 / 4
T_2 	3 / 1 	5 / 1	1 / 3	2 / 5	4 / 3
T_3 	5 / 2 	4 / 3	2 / 2 	1 / 4 	3 / 2 
T_4 	1 / 4	3 / 2	5 / 4	4 / 3	2 / 1 
T_5 	4 / 3	2 / 5	3 / 1 	5 / 2	1 / 5

MiniZinc Features

MiniZinc Type System

- Basic types:
 - `int`, `bool`, `float`
- Enumerated types: `enum`
 - named discrete (ordered) types.
- Set types: `set of <T>`
 - **Only** sets of `int`'s/`enum`'s are allowed to be decision variables
- Array types: `array [<IndexSet1>, <IndexSet2>, ...] of <ElementType>`
 - Arrays have dimension sizes **fixed** by the input

Beware solvers make mistakes using floats

MiniZinc Type System

- Extended enumerations
 - `enum OP = DUP(1..m) ++ SWAP(2..m) ++ {POP, ADD, MUL};`
 - `enum XOP = 0(OP) ++ { NOP };`
 - Constructor function `0` and (partial) inverse `0^-1` are generated too.
- Tuples and Records
 - `type Point = record(0..infinity: x, 0..infinity: y);`
 - Note also type synonyms
- Strings: `string`. No string decision variables, only for output processing/debugging.

Strong Typing

- We encourage modellers to use **strong types**:
 - enums, extended enums, records
- Hidden type errors in optimisation problems can be **very difficult** to debug.
 - The solver may simply say UNSATISFIABLE,
 - Or **worse** run for a long time with no answer
- We are adding more strong type features: **unit types**

Variables

- Parameters: variables with a value fixed by the input

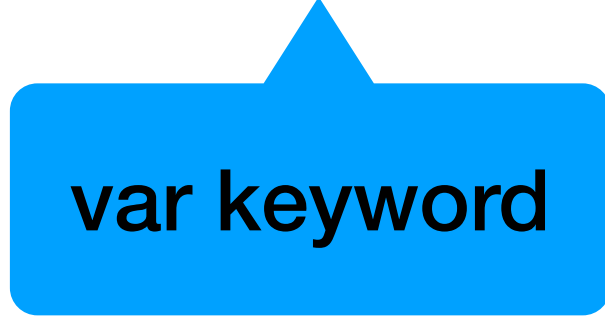
- `enum TASK;`

- `array[TASK] of int: duration;`

- `par int: makespan = sum(duration);`  Par optional; never actually used!

- Decision variables: variables to be decided by the solver

- `array[TASK] of var 0..makespan: start_time;`



var keyword

Separating Model and Data

- Models are generic in input data (parameters)
- Data file + model together specify an **instance**

Model

```
enum TASK;  
array[TASK] of int: duration;  
int: makespan = sum(duration);  
array[TASK] of var 0..makespan: start_time;  
constraint disjunctive(start_time,duration);
```

Data

```
TASK = { T1, T2, T3, T4 };  
duration = [ 14, 12, 6, 8 ];  
%makespan will be calculated as 40
```

Constraints

- Arithmetic operators: $+$, $-$, $*$, $^$, `div`, `mod`, `abs`, `min`, `max`, ...
- Arithmetic relations: $=$, $<$, $\leq$, $\neq$.
- Boolean operators: $/\wedge$, $\wedge/$, $\rightarrow$, $\leftrightarrow$, `xor`, `forall`, `exists`, ...
- Constraints: `constraint` `<BoolExpr>`;
- Example
- `constraint` $y \geq x \wedge y \geq -x \wedge (y = x \wedge y = -x)$;

Array

- Range notation: $1 \dots u$ defines the set $\{ d \mid 1 \leq d \wedge d \leq u \}$
- (Multidimensional) Arrays have range index sets, e.g.
 - `array[TASK, 0..makespan] of var bool: executing;`
- Indices are part of the array type, so e.g.
 - `array[0..n-1] of var int: x;`
 - `array[1..n] of var int: y = array1d(x); n, x); smatch`

Default index set: 1..length(x)

Array coercion

Array Slicing

- You can generate slices of arrays
 - e.g. `executing[t, ..]` is a array of var bool with index `0..makespan`
 - `executing[TASK, 0]` is an array of var bool with index `TASK`
 - `executing[t1..t3, 1..makespan-1]` is a 2D array with indices `t1..t3, 1..makespan-1`
- Slices need to be **fixed** at compile time

Arrays

- MiniZinc arrays must be **fixed** in size by the input
- They support **decision variables** as indices (like the Theory of Arrays)
- Variable indices avoid the **many indices** problem of 01 MIP, SAT
 - E.g. For each task t we need to choose a machine m and a recipe r
 - Standard 01 modelling: $x_{tmr} = 1$ if task t runs on machine m with recipe r
 - MiniZinc formulation: `machine[t] = m /\ recipe[t] = r`

Arrays

- CP and MIP solvers will reason “disjunctively” about MiniZinc arrays
- e.g. $x = [3, 5, 3, 8, 16, y][i]$, $i \in \{1, 2, 6\}$, $y \in \{4, 5, 10\}$
 - CP solver will determine $x \in \{3, 4, 5, 10\}$
 - MIP solver will determine $x \in 3..10$
- This makes arrays a **key modelling tool** for efficient models

Comprehensions

- The only “looping” construct in MiniZinc are **comprehensions**
- Python: `[ 2 * x for x in range(20) if x*x > 3 ]`
- Haskell: `[ 2 * x | x <- [0..19], x^2 > 3 ]`
- **MiniZinc**: `[ 2 * x | x in 0..19 where x^2 > 3 ]`
- General form: `[ <expr> | <generator> where <test>, ... ]`
- Example: `[ duration[t] | t in TASK where executed[t] ]`
- But we “hide” comprehensions in **generator expressions**

Generator Expressions

- The syntax `<function/predicate> (<generator> where <test>) (<expr>)`
- is syntactic sugar for
 - `<function/predicate> ( [ <expr> | <generator> where <test> ] )`
- For example,
 - `forall(t in TASK) (start[t] + duration[t] <= jobend);`
- Is really
 - `forall([ start[t] + duration[t] <= jobend | t in TASK] );`
- i.e. build a list of Boolean variables, then enforce they are all true

Generator Expressions

- Commonly used functions/predicates
 - `forall`:
 - `exists`:
 - `sum`:
- But we can also use arbitrary functions and predicates
 - `alldifferent(i in ROW) (q[i] + i)` is equivalent to
 - `alldifferent([ q[i] + i | i in ROW ])`

Comprehensions Bonus Feature

- Comprehensions (and generator expressions) can
 - Iterate over **variable sets** (sets not known at compile time)
 - Use **variable where** conditions

- For example,

```
array[TASK] of var bool: executed; % is task executed?  
var int: time_used =  
    sum( [ duration[t] | t in TASK where executed[t] ] );
```

- Implemented by option types (which we will elide today)



Decision variable

Conditionals

- MiniZinc allows conditional expressions of any type, e.g.
 - `if <BExpr> then <TExpr> [elseif ... ] else <EExpr> endif`
- Where <TExpr> and <EExpr> have the same type, <BExpr> is Boolean
- Bonus: the <BExpr> need not be **known at compile time**, if the type of <TExpr> is *varifiable* (not containing arrays of unfixed size or sets of floats)
- If-then-else-endif is a **very powerful** modelling tool
 - But its **overused** by procedural programmers

Quick MiniZinc Demo 2

- Single Constant Multiplication (Thanks to V. Lagoon + A. Metodi)
- Imagine you want to compute: $y = k \times x$, k is constant, x in binary
- Naive approach: $y = \sum_{i \in \text{bits}(k)} 2^i \times x$
 - $2^i \times x$ is a shift, so this requires $|\text{bits}(k)| - 1$ additions
- Clever approach: $k = 99 = 0b1100011$ (naive requires 3 additions) but
 - $2^1 \times x + x = 3 \times x$
 - $2^5 \times (3 \times x) + (3 \times x) = 99 \times x$

Quick MiniZinc Demo 2

- Single Constant Multiplication (Thanks to V. Lagoon + A. Metodi)
- Note: we don't have to create **even** multiples (just shift the result)
- Three “recipes”
 - PLUS: $2^{shift} \times left + right = new$
 - MINUS: $2^{shift} \times left - right = new$
 - MINUSP: $left - 2^{shift} \times right = new$
- Search for the quickest recipe for each **odd** constant k

Saved the best until last

Let expressions

- **Let expressions** allow the introduction of new parameters or decision variables and even constraints “anywhere”

- Format: `let { [ <Decl> | <Constraint> ]* } in <Expr>`

- Example

```
forall(i in 1..n-1)(let { var 0..n: t;  
                        constraint y[i] = 2*t; } in  
y[i+1] >= t /\ y[i+1] <= y[i] + t);
```

Local decision variable

Constrained to equal half y[i]

Used in the body expression twice

Let expressions: restrictions

- Let expressions that are in negated or “mixed” positions cannot introduce new decision variables (unless they are functionally defined)

```
constraint not (let { var int: t;  
                    constraint a = 2*t; } in t > b);
```

Local decision variable in
negated context

- This can implicitly “universally” quantify variables
 - Not supported by solvers
- For this example there is an alternate valid expression

```
constraint not (let { var int: t = a div 2;  
                    constraint a mod 2 = 0; } in t > b);
```

Local decision variable
functionally defined

Let expressions: semantics

- Constraints in let expressions “float up” to the nearest enclosing Boolean expression

- Example

```
constraint not (let { var int: t = a div 2;  
                    constraint a mod 2 = 0; } in t > b);
```

- Is equivalent to

```
constraint not (a mod 2 = 0 /\ let { var int: t = a div 2; } in t > b);
```

- Not equivalent to

```
constraint a mod 2 = 0 /\ not (let { var int: t = a div 2; } in t > b);
```

Predicates (and functions)

- MiniZinc allows you to define your own predicates and functions
- For example, a disjunctive scheduling predicate requiring no two tasks overlap in executions

```
predicate disjunctive(array[int] of var int: s, array[int] of int: d) =  
    forall(t1, t2 in index_set(s) where t1 < t2)  
        (s[t1] + d[t1] <= s[t2] \/\ s[t2] + d[t2] <= s[t1]);
```

Reflection functions allow us to
determine index sets and bounds

- Predicates are just functions returning `var bool`!
- Let expressions allow us to include new vars and constraints in functions

Global constraints

- One of the key advantages of the Constraint Programming view of modelling
- **Global constraints:** Predicates encapsulating common combinatorial substructures
 - Scheduling + Packing: `disjunctive`, `cumulative`, `bin_packing`
 - Counting: `global_cardinality`, `count`, `distribute`
 - Graphs: `dag`, `tree`, `connected`, `circuit`
 - Functions: `alldifferent`, `inverse`
 - Extensible: `table`, `regular`, `mdd`
 - Symmetry: `lex_lesseq`, `value_precede_chain`

Global constraints

- Global constraints support
 - Concise models
 - Specialised decompositions for different solvers
 - Including for CP/SMT solvers specialised algorithms to solve them

- Example: `alldifferent` for MIP

```
predicate alldifferent(array[int] of var int: x) =  
    forall(d in dom_array(x))  
        (sum(i in index_set(x))(x[i] = d) <= 1);
```

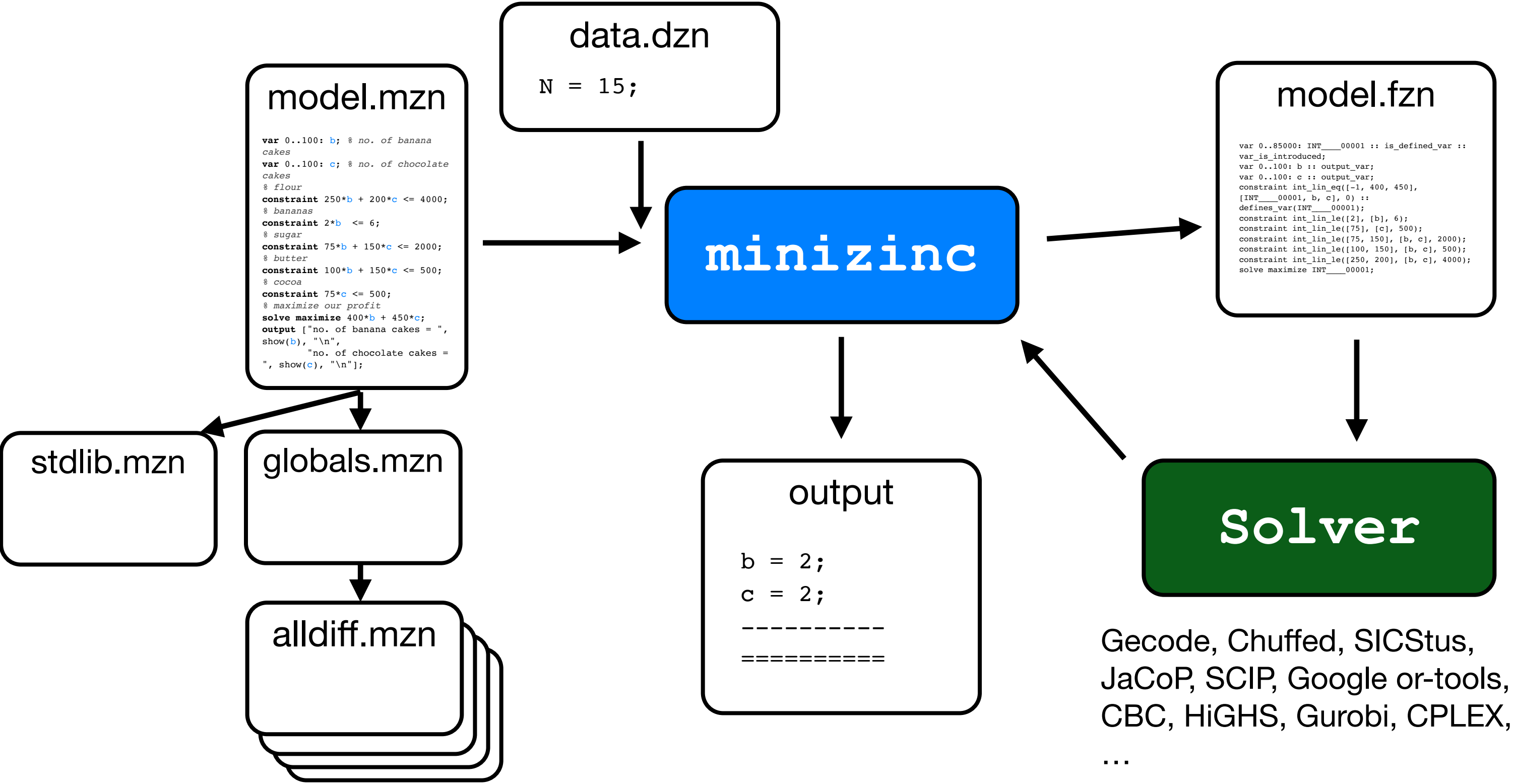
- Example: `alldifferent` for CP

```
predicate alldifferent(array[int] of var int: x);
```

No definition: sent directly to solver

Compiling MiniZinc

Compiling MiniZinc



Example translation

```
predicate two_apart(var int: i, var int: j) =  
    i + 2 = j \/\ j + 2 = i;  
  
var 1..5: a;  
var 3..6: b;  
  
constraint two_apart(a, b);
```

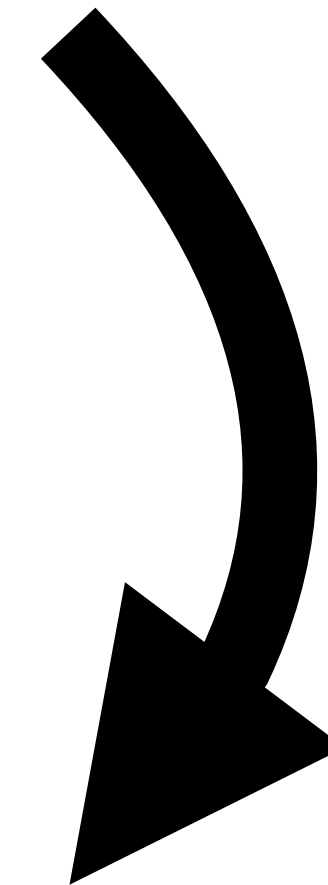
Example translation

```
predicate two_apart(var int: i, var int: j) =  
    i + 2 = j \/\ j + 2 = i;
```

```
var 1..5: a;  
var 3..6: b;
```

```
constraint two_apart(a, b);
```

```
var int:    a = mk_intvar(1..5);  
var int:    b = mk_intvar(3..6);  
constraint  two_apart(a, b);
```



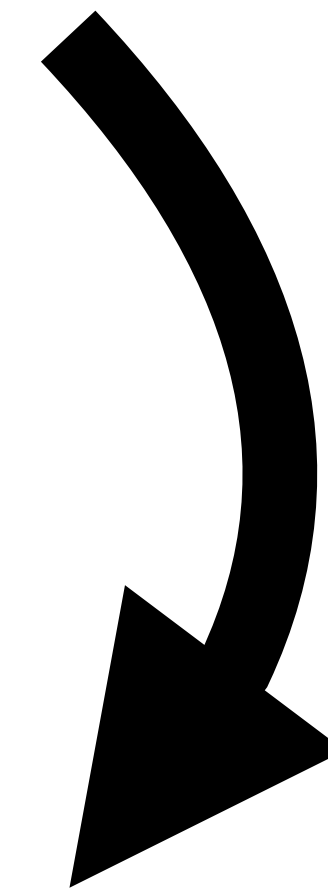
Example translation

```
predicate two_apart(var int: i, var int: j) =  
    i + 2 = j \ / j + 2 = i;
```

```
var 1..5: a;  
var 3..6: b;
```

```
constraint two_apart(a, b);
```

```
var int:    a = mk_intvar(1..5);  
var int:    b = mk_intvar(3..6);  
constraint  bool_or(i1, i2);  
var bool: i1 = int_plus(a, 2, b);  
var bool: i2 = int_plus(b, 2, a);
```



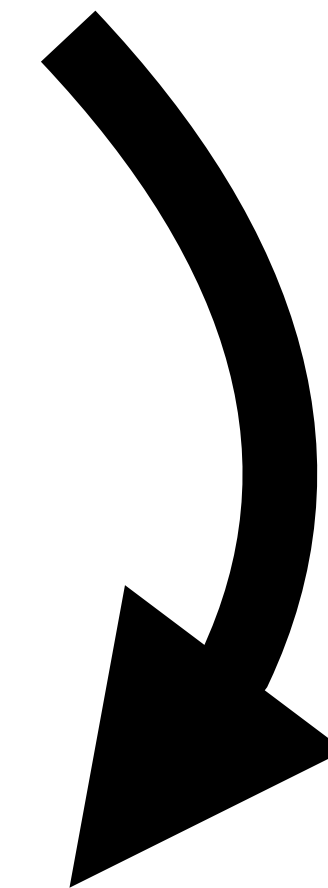
Example translation

```
predicate two_apart(var int: i, var int: j) =  
    i + 2 = j \ / j + 2 = i;
```

```
var 1..5: a;  
var 3..6: b;
```

```
constraint two_apart(a, b);
```

```
var int:    a = mk_intvar(1..5);  
var int:    b = mk_intvar(3..6);  
constraint  bool_or(i1, i2);  
var bool: i1 = mk_boolvar();  
var bool: i2 = mk_boolvar();  
constraint  int_plus_reif(a, 2, b, i1);  
constraint  int_plus_reif(b, 2, a, i2);
```



suitable for a CP solver
(e.g. Gecode)

Compiler or Interpreter?

Modelling language point of view:

We compile a MiniZinc specification into an equivalent FlatZinc specification

Programming language point of view:

We execute the MiniZinc program, and that results in the output of a FlatZinc specification

In reality, it's a mix of the two. We specify **both the model and the translation** to the target solver language in MiniZinc.

The MiniZinc compiler / interpreter

- Model is **parsed** into an Abstract Syntax Tree (AST)
- Interpreter recursively evaluates each expression
 - evaluation returns a **result** (value and whether it is defined)
 - evaluation may **emit constraints**
 - e.g. evaluating `x+y` may return a fresh variable `X_INTRODUCED_0` and emit constraint `int_plus(x, y, X_INTRODUCED_0)`
 - e.g. evaluating `x div y` may return a fresh variable `X_INTRODUCED_0` for the value and another (Boolean) variable `X_INTRODUCED_1` that is true iff `y≠0`
- Performs **Common Subexpression Elimination**
- Performs **Constraint Propagation** (to simplify generated FlatZinc)

Example translation

user-defined predicate

```
predicate two_apart(var int: i, var int: j) =  
    i + 2 = j \ / j + 2 = i;  
  
var 1..5: a;  
var 3..6: b;  
  
constraint two_apart(a, b);
```

“built-in” solver predicates

```
var int: a = mk_intvar(1..5);  
var int: b = mk_intvar(3..6);  
constraint bool_or(i1, i2);  
var bool: i1 = mk_boolvar();  
var bool: i2 = mk_boolvar();  
constraint int_plus_reif(a, 2, b, i1);  
constraint int_plus_reif(b, 2, a, i2);
```

“built-in” solver predicates

What's a built-in?

- Any predicate that **doesn't have a body** is considered a built-in.
- All predicates are defined in either **user code** or the **MiniZinc library**.
- If a target solver doesn't support a particular built-in, simply **redefine it** in terms of built-ins it does support!

Example redefinition: reification for MIP

$$\begin{aligned} & b \rightarrow x \leq y \\ \equiv & b \rightarrow (x - y) \leq 0 \\ \equiv & (x - y) \leq (1 - b) * M \\ \equiv & (x - y) \leq (1 - b) * (\text{ub}(x) - \text{lb}(y)) \end{aligned}$$

declared variable bounds

```
% b -> x <= y  
predicate int_le_imp(var int: x, var int: y, var bool: b) =  
    (x - y) <= (1 - b) * (ub(x) - lb(y));
```

Modelling State in MiniZinc

Modelling state: traditional

- Model the state at each point in execution
- Essentially write code to determine the new state $s[t]$ from the previous state, $s[t-1]$
- Example: simple stack language: DUP(i), SWAP(i), POP, OP

```
array[TIME0, 1..m] of var TERM: s; % note bounded stack representation 1..m
array[TIME] of var OPCODE: o;
    . . .
elseif o[t] = POP then
    s[t,..] = s[t-1,2..m] ++ [ nil ] % add empty value at end
else % NOP
    s[t,..] = s[t-1,..]
endif);
```

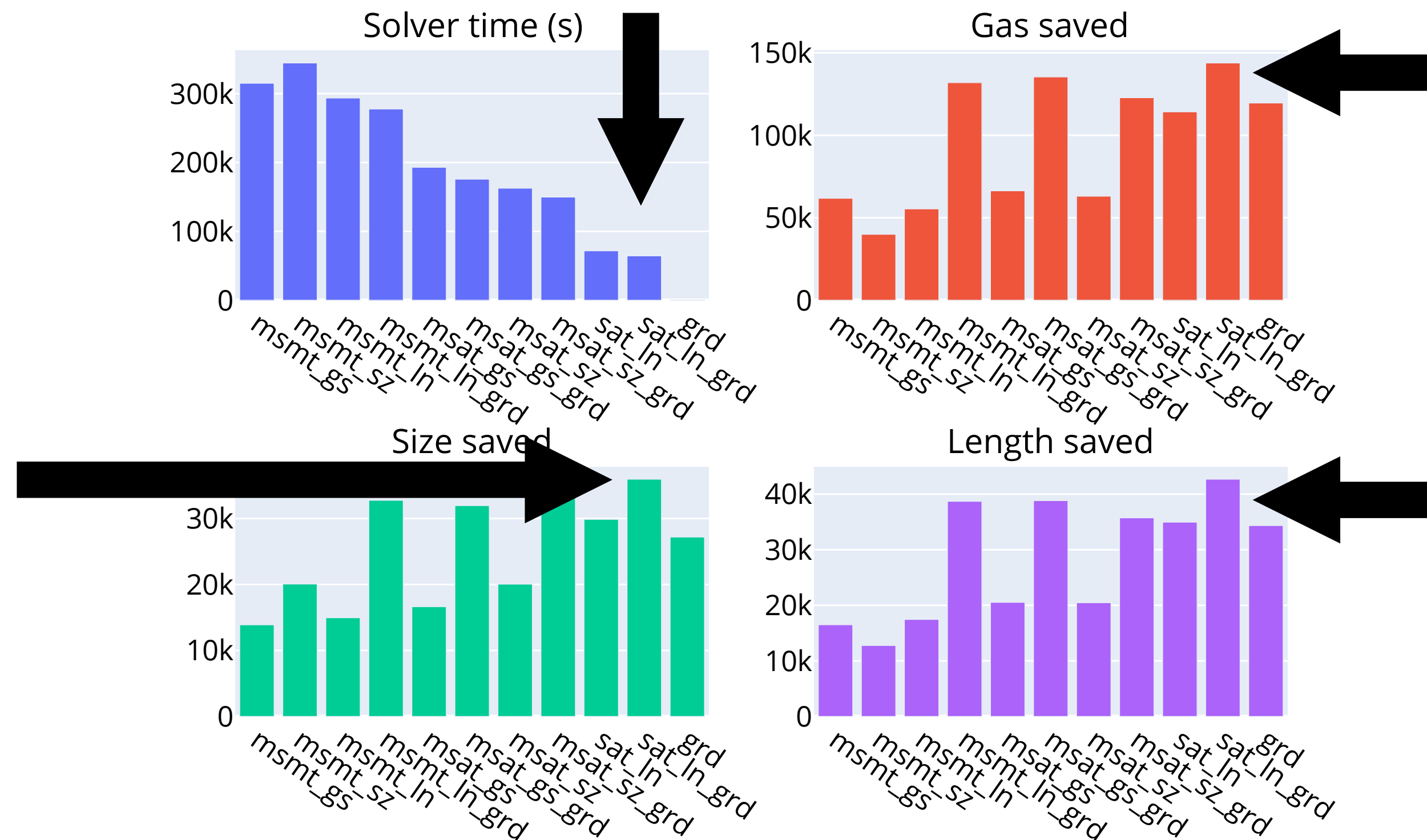
- Worth also adding constraints to propagate information backwards

Modelling state: traditional

- We recently used this approach for Ethereum code **super optimisation**
- Built an entire model in MiniZinc to solve the problem
 - Explored different ways of modelling the key decisions
 - Explored how to “best” define constraints
 - A first prototype in **1 day**
 - Explored which redundant constraints sped things up
 - Explored which (almost) dominance constraints helped
- Finally took the **best combination**, and built a direct SAT model (in **two weeks**)
 - The direct SAT model is around **4 times faster** than MiniZinc/chuffed

Modelling state: traditional

- MiniZinc and the final SAT model were able to
 - Substantially reduce program length, size and gas (cost to execute)



Modelling state: alternative

- Rather than model explicit state at each point
- Model which previous assignment gives the value being referred to
- Consider looking up the value of `Veg.fullpizzas` at line 30
- Previous assignments that could change this value are

(Unr.)Line	Condition	Object	Field	Value
1		Veg.	fullpizzas=	0
18	(b1)	pizza1.	fullpizzas=	pizza1.fullpizzas+1
23	(b2)	pizza2.	fullpizzas=	pizza2.fullpizzas+1

- How do we compute the

Modelling state: alternative

- Constraints to determine value of `Veg.fullpizzas` at line 30

(Unr.)Line	Condition	Object	Field	Value
1		Veg.	fullpizzas=	0
18	(b1)	pizza1.	fullpizzas=	pizza1.fullpizzas+1
23	(b2)	pizza2.	fullpizzas=	pizza2.fullpizzas+1

- Which assignment `i` led to this value

```
[Veg,pizza1,pizza2][i] = Veg /\ % agreement on the object
[true,b1,b2][i] = true /\ % statement was executed
[0, pizza1_fullpizzas_l18 + 1, pizza2_fullpizzas_l23 + 1][i] = Veg_fullpizzas_l30 /\
(b1 /\ pizza1 = Veg -> i >= 2) /\
(b2 /\ pizza2 = Veg -> i >= 3)
```

Principle advantage: partial information flow before `i` is known

Modelling state: alternative

- At least for some applications
 - e.g. mapping simulation code to optimisation code
- Alternative encoding was
 - Substantially smaller and more effective than traditional CP approach
 - Substantially more effective than SMT (theory of arrays) approach

Conclusion

MiniZinc



- MiniZinc is a defacto standard for Constraint Programming modelling
- MiniZinc can be used with:
 - CP, MIP, SAT, SMT, ASP, and local search solvers
- MiniZinc is widely used (probably mainly by MIP modellers 😞)
 - In **industry** and **academia well outside Computer Science**
- MiniZinc is pretty **robust**, and we hope we are reasonably **responsive**
- MiniZinc is actively being **developed** (e.g. tuples are new this year)

MiniZinc to SAT



- You can already convert MiniZinc models to SAT
 - Build the MiniZinc model, use the PICAT-SAT solver, and instruct it to output CNF
- We are working on a **direct interface**: MiniZinc to SAT/MaxSAT
 - Control of integer encodings
 - Automatic or controlled constraint encodings (given integer encodings)
 - Automatic coercion between different encodings
- Hopefully released next year!

MiniZinc to SMT



- You can already use SMT/OMT as backends to MiniZinc
 - **fzn2smt**: tool developed by Miquel Bofill, Miquel Palahí, Josep Suy, and Mateu Villaret, Carlos Ansótegui in **2011**
 - **fzn2omt**: tool developed by Patrick Trentin in **2023**
- We are happy to help you develop new ones!

Towards MiniZinc 3.0



The current interpreter

- Struggles when generating **large output models**
- Has no support for **incremental** translation and solving

We're working on a **complete redesign**

- Add a compilation step to make interpretation more efficient
- Optimised interpreter and internal data structures
- Fully incremental (add and retract constraints)

2024/2025 release!

Summary



- MiniZinc is a **generic translator** + backend-specific **library**
- **Encodings are defined in MiniZinc itself**
- MiniZinc is adept at describing finite domain (optimisation) problems
 - Which includes many formal reasoning problems
- Reasoning about state using CP can be different



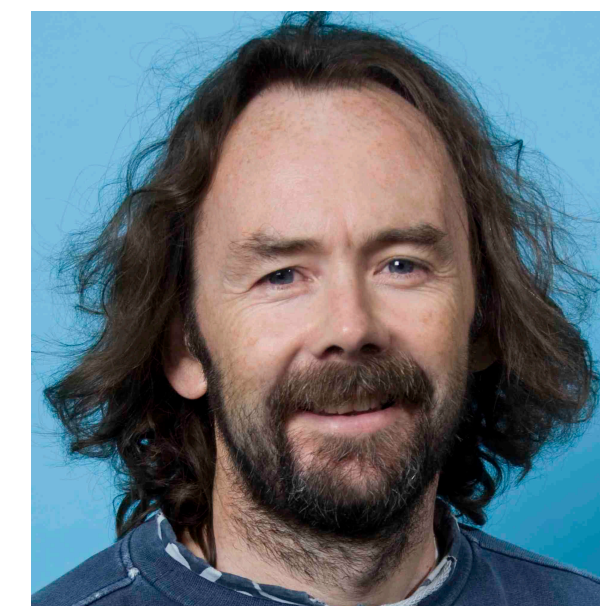
Jip J. Dekker



Jason Nguyen



Hendrik Bierlee



Peter J. Stuckey



Guido Tack

FEDERAL

GALAXY

TOP NEWS

ENLIST

EXIT



✚ WOULD YOU LIKE TO KNOW MORE?

MiniZinc Resources



- Coursera Courses (Free if you don't want a certificate)
 - Basic Modelling for Discrete Optimization
 - Advanced Modelling for Discrete Optimization
 - Solving Algorithms for Discrete Optimization
 - [coursera.org](https://www.coursera.org)
- MiniZinc Resources
 - Tutorial: https://www.minizinc.org/doc-2.7.6/en/part_2_tutorial.html
 - Reference manual: https://www.minizinc.org/doc-2.7.6/en/part_4_reference.html
- MiniZinc Book
 - Building Decision Support Systems: using MiniZinc. Mark Wallace, Springer 2020